Airflow 2.10.5 to 3.3.2 Migration Plan (Blue/Green on AKS)

Oct 1, 2026 · @Surya

1. Executive summary

Each environment (dev, UAT, production) moves from Airflow 2.10.5 to 3.3.2 by building a fully independent v3 stack beside v2, migrating a clone of the metadata database, and switching traffic at a planned cutover. Airflow v2, its database, its Redis and its DAG share are never modified, so rollback is: stop v3, start v2, switch traffic back.

Item	Decision
Approach	Blue/green (Plan B), repeated in every environment so each cutover rehearses the next
v3 placement	Separate namespace on a separate AKS nodepool
Metadata DB	New database holding a migrated clone of v2′s data; v2 DB stays untouched
Broker	New Redis instance (Celery broker)
DAG and log storage	New Azure Files share (SMB, existing CSI driver) mounted only by v3
DAG delivery	Existing Nexus release process, deploying the same released version to both shares
Rollback	Scale v3 to zero, start v2, repoint traffic; target under 30 minutes
Out of scope (Phase 2)	NFS 4.1 premium share and Azure Blob remote logging
Migration path	Direct 2.10.5 to 3.3.2 on a cloned database; a hop through 2.11.x only if proven necessary
Python	v3 runs Python 3.13; v2 stays on Python 3.11, so DAG releases must run on both

What success looks like.

- Zero data loss in the v2 database and zero changes to v2 pods, secrets or storage.
- All tier-0 and tier-1 DAGs run on v3 with equal or better success rates than the v2 baseline.
- A rehearsed, timed rollback exists before any production cutover is approved.

Why Airflow 3 is a larger change than a normal upgrade. Airflow 3.x introduces new components (api-server, standalone dag-processor), a new execution model for tasks, and removed or renamed DAG features. Source: the official upgrade guide. Airflow 3.3.2 was released on 17 September 2026 (release page).

Indicative duration. About 9 to 10 weeks from kickoff to production cutover, plus a 2-week stabilization window before v2 is retired (section 4).

2. Current and target architecture

The v3 stack keeps your platform shape (AKS, Celery, Redis, PostgreSQL 17, Azure Files) but replaces the webserver with an api-server, adds a standalone dag-processor, and changes how workers reach the metadata database.

Area	Airflow 2.10.5 today	Airflow 3.3.2 target	Impact on us
UI and API	Webserver (Flask); REST API at `/api/v1`	api-server (FastAPI); REST API at `/api/v2`; new UI	External API callers and health probes must change
DAG parsing	Scheduler-managed	Standalone `dag-processor` process, required	New pod, needs the DAG mount

Area	Airflow 2.10.5 today	Airflow 3.3.2 target	Impact on us
Task to DB access	Workers talk to the metadata DB directly	Workers use the Task SDK and the api-server execution API	Workers need network access to the api-server; DAG code must not touch the DB directly
Authentication	FAB RBAC built in	Pluggable auth manager; FAB moved to a separate provider	Needs the FAB provider if you use RBAC, LDAP or OAuth today
Executor and broker	CeleryExecutor with Redis	Unchanged	New Redis instance for isolation
Core operators	In core	In the standard provider	Included in the official image; pin versions
Scheduling defaults	`catchup_by_default=True`; cron data intervals on	`catchup_by_default=False`; cron data intervals off	Review every DAG's schedule semantics

Area	Airflow 2.10.5 today	Airflow 3.3.2 target	Impact on us
DAG code changes	`execution_date`, SubDAGs, SLAs, `schedule_interval`, direct session use	Removed or changed	Ruff AIR rules plus manual review (section 5)
DAG versioning	None	DAG bundles with versioning	Local bundle over the Azure Files share keeps today's behavior
Task logs	Served through the webserver	Served through the api-server	Logs volume must be mounted where the api-server runs

Pods per stack.

v2 (stays as is, scaled to zero after cutover)	v3 (new)
webserver	api-server
scheduler	scheduler
(none)	dag-processor
triggerer	triggerer
celery workers	celery workers

Isolation boundary. Nothing is shared between the two stacks except the PostgreSQL server (if you choose a new database on the same server) and the cluster itself. Section 3 and appendix A define the full list of separated resources.

The scheduling, dag-processor and worker behaviors above come from the Airflow 3 upgrade guide and release notes; confirm each against 3.3.2 during the dev build (appendix D).

3. Design decisions and principles

Four principles drive every step: v2 is never touched, only one stack is live at a time, every irreversible action happens on a copy, and every cutover is rehearsed with timings before it is attempted.

ID	Decision	Rationale	Reversible?
D1	Blue/green per environment, with the same pattern in dev, UAT and prod	Each lower-environment cutover rehearses the production runbook	Yes
D2	Migrate a clone of the v2 metadata DB; no clean install	Keeps run history, connections, variables, pools and scheduling state, so catchup does not misfire	Yes, v2 DB untouched
D3	Reuse the v2 Fernet key in v3	Stored connection passwords and variables must still decrypt	Yes
D4	New JWT secret and API secret key for v3	These are new in Airflow 3; do not reuse anything from v2	Yes
D5	New Redis instance	v2 queue messages are not valid for v3 workers; guarantees an empty queue	Yes
D6	New Azure Files share and PV/PVC for v3 (SMB, existing CSI driver)	Keeps v2 DAGs and logs intact; no storage-type change during the upgrade	Yes
D7	Nexus-released DAG version deployed to both shares during the parallel period	Meets your release rules; makes DAG rollback a redeploy of an older version	Yes

ID	Decision	Rationale	Reversible?
D8	DAG releases stay compatible with v2 (Python 3.11) and v3 (Python 3.13) until v2 is retired	A rollback must not require a DAG code rollback	Yes
D9	Controlled start: all DAGs paused in v3 until smoke tests pass, then unpaused in tiers	Avoids a burst of catchup or missed-interval runs at startup	Yes
D10	Direct 2.10.5 to 3.3.2 migration is the default; hop through 2.11.x on the clone only if proven necessary	Direct is the preferred path; the upgrade guide recommends going via the latest 2.x, so dev must prove direct works. Any hop runs on the clone only, never on v2	Yes
D11	Separate hostname during testing; production hostname switches at cutover through DNS or ingress	Reversible in seconds with a low TTL	Yes
D12	Change freeze on DAG releases from T-48h through the end of hypercare	A v3-only DAG change would break rollback	Yes
D13	Keep v2 stack, database and share for at least 14 days after cutover	Defines the rollback window and then the fix-forward point	n/a
D14	Python 3.13 in the v3 image; v2 stays on Python 3.11	Moves to a current supported runtime in the same migration; v2 keeps 3.11 so rollback is untouched, which means DAG releases must run on both	Yes

Rules that follow from these decisions.

- No shared Redis, no shared file share paths, no shared secrets, no shared Celery result backend between v2 and v3.
- The v3 `result_backend` must point to the v3 database, or v3 writes Celery tables into the v2 database.

- v2 and v3 schedulers must never run at the same time against the same business DAGs.
- Every command that changes data runs against the v3 database or the clone, never the v2 database.

4. Roles, governance and schedule

Production cutover needs one named decision-maker and one named rollback authority; these are usually the same person. Names below are role placeholders to fill in at kickoff.

Role	Responsibility	Decides
Change owner (platform lead)	Owns the plan, runs the bridge, calls go/no-go	Go, no-go, rollback
Airflow platform engineers	Helm, images, secrets, runbook execution	Technical steps
AKS / cloud engineers	Nodepool, quota, networking, storage, Redis	Infrastructure readiness
DBA / data platform	PostgreSQL backups, clone, restore, timing	DB readiness
DAG owners / product teams	DAG fixes, tier classification, test sign-off	DAG readiness per tier
Release / DevOps	Nexus pipeline targets for both shares, change freeze	Release readiness
Security	Auth manager, secrets, network policy review	Security sign-off
Operations / support	Monitoring, hypercare rota, user comms	Hypercare exit

DAG tiers. Classify every DAG before testing starts, since tiers drive test depth, unpause order and rollback reconciliation.

Tier	Definition	Handling
0	Business-critical or customer-facing; must not miss or duplicate runs	Full regression, owner present at cutover, unpaused first

Tier	Definition	Handling
1	Important internal pipelines	Regression on representative runs, unpaused second
2	Everything else	Import and smoke test, unpaused last

Indicative schedule (weeks from kickoff; adjust to your change calendar).

Weeks	Activity	Environment	Exit criterion
1-2	Phase 0 readiness: inventory, Ruff scan, provider and plugin review, Python 3.13 dependency check, image build	Local / dev	Image builds; list of DAG fixes agreed
2-4	DAG fixes released (v2 and v3 compatible) to the existing v2 environments	Dev, UAT, prod (v2)	Fixes live on v2 with no regressions
3-4	Build v3 stack, clone, migrate, test, cutover, rollback drill	Dev	Dev signed off; runbook v1 corrected
5-6	Same pattern with production-like data and full test matrix	UAT	UAT sign-off; timed cutover and timed rollback drill
7-8	Build v3 stack, dress rehearsal on a fresh clone of prod DB	Prod (v3 idle)	Rehearsal timings fit the window; go/no-go 1
9	Production cutover	Prod	Go/no-go 3 at T+4h and T+24h
9-11	Stabilization window; v2 kept on standby (scaled to zero)	Prod	No Sev1 for 7 days and no open Sev2 at day 14
11+	Decommission v2; start Phase 2 planning	Prod	Section 13 checklist complete

Change windows. Pick a production window that fits the measured migration time from the UAT and rehearsal runs, plus at least 2 hours of validation and a rollback reserve equal to the time of the cutover itself. Avoid month-end and any window where tier-0 DAGs have tight SLAs.

5. Phase 0: readiness (before any v3 infrastructure)

The goal of Phase 0 is to make the DAG estate, configuration and image ready for v3 while everything still runs on v2. DAG fixes ship to v2 first, so the platform change and the code change never land together.

5.1 Inventory

- [ ] Export the list of all DAGs with owner, tier, schedule, `catchup`, `start_date`, pool and last 90-day success rate.
- [ ] List every installed provider and version (`pip freeze` from the v2 image).
- [ ] List custom plugins, custom operators and hooks, `airflow_local_settings.py`, cluster policies and DAG-level callbacks.
- [ ] List every external caller of the Airflow REST API (CI/CD, monitoring, other apps) and how each authenticates.
- [ ] List connections, variables and secrets backends in use, and where the Fernet key lives.
- [ ] Record the current auth setup (FAB RBAC roles, LDAP/OAuth config, custom roles) and the user and role list.
- [ ] Capture v2 baselines: DAG parse time, scheduler loop time, task queue latency, success rate, worker CPU and memory.

5.2 DAG compatibility

1. Install a current Ruff and run the Airflow rules on the DAG repository:

   ```
   pip install --upgrade ruff
   ruff check --preview --select AIR30 dags/
   ruff check --preview --select AIR30 dags/ --show-fixes
   ```

   AIR301 and AIR302 flag removals that break on Airflow 3; AIR311 and AIR312 flag changes that are recommended but not yet breaking (upgrade guide).
2. Fix everything flagged by AIR301 and AIR302, choosing replacements that also work on 2.10.5. Defer AIR311/AIR312 modernizations (such as imports from the new SDK package) until v2 is retired, because those may not exist on v2.
3. Review items Ruff cannot catch:
   - Direct metadata DB access from tasks (settings sessions, `provide_session`, raw ORM queries); move these to supported APIs or hooks.

- Use of `execution_date`, SubDAGs, SLAs, `schedule_interval`, `provide_context`, `days_ago`, `airflow.contrib` imports.
- Reliance on `catchup=True` defaults, cron-based data intervals, or implicit `start_date` behavior; set these explicitly on every DAG.
- Callbacks, `on_failure` handlers and templates that use removed context keys.
- Custom XCom backends or pickled XComs.

4. Release the fixes through your normal Nexus process to dev, UAT and then v2 production, and let them run for at least one full business cycle.

5.3 Providers, plugins and configuration

- [ ] Resolve provider versions that support Airflow 3.3.2, using the project's constraints file for 3.3.2; remove unused providers.
- [ ] Add the FAB provider and set the FAB auth manager if you use RBAC, LDAP or OAuth; otherwise decide on the replacement auth manager and map roles.
- [ ] Re-test custom plugins; web UI plugins written for the Flask UI need the FAB provider or a rewrite for the new UI.
- [ ] Run the configuration linter on your v2 configuration, then apply fixes:

```
airflow config lint
airflow config update
```

- [ ] Move the runtime from Python 3.11 (v2) to Python 3.13 (v3). Airflow 3.3.2 supports 3.10 to 3.14, so the work is making DAG code and dependencies run on 3.13 (section 5.5).
- [ ] Confirm PostgreSQL 17 and the Celery and Redis provider versions are supported by 3.3.2.

5.4 Image build

- [ ] Build one immutable v3 image from the official `apache/airflow:3.3.2` base, installing extras with the 3.3.2 constraints file: `https://raw.githubusercontent.com/apache/airflow/constraints-3.3.2/constraints-<python-version>.txt`.
- [ ] Install no DAGs in the image; DAGs come from the share (your no-rebuild requirement).
- [ ] Scan the image, push it to your registry, and reference it by digest, not by tag.

- [ ] Use the same image for the scheduler, api-server, dag-processor, triggerer, workers and migration job.

5.5 Python 3.11 to 3.13

The runtime moves from Python 3.11 to 3.13 in the same upgrade. Airflow 3.3.2 supports Python 3.10 to 3.14, so 3.13 is supported; the work is making DAG code, custom libraries and third-party dependencies run on 3.13. While v2 is alive they must also keep running on 3.11.

- **Two runtimes, one DAG release.** v2 keeps its Python 3.11 image and v3 runs 3.13, and the same Nexus release lands on both shares. Rollback depends on this, so a release that only works on 3.13 is not allowed until v2 is retired.
- **Language rules in the parallel period.** Do not use syntax or standard-library features that exist only in Python 3.12 or later (for example `type` statements and the new generic class syntax).
- **Removed standard-library modules.** Python 3.12 removed `distutils`, `imp`, `asyncore`, `asynchat` and `smtpd`. Python 3.13 removed the modules retired by PEP 594, including `cgi`, `cgitb`, `crypt`, `pipes`, `telnetlib`, `nntplib`, `imghdr`, `uu` and `xdrlib`, plus `lib2to3`. Search DAG, plugin and library code for these imports.
- **Dependencies.** Take `pip freeze` from the v2 (3.11) image and resolve every package against the 3.3.2 constraints file for Python 3.13 (`constraints-3.3.2/constraints-3.13.txt`). Check that packages with native code (for example numpy, pandas, pyarrow, grpc, psycopg2, cryptography) have 3.13 wheels, so the image build does not compile from source.
- **Version drift.** Airflow's constraints may pin different versions of libraries your DAGs also use. Treat each version change as a behavior change and cover it in regression tests.
- **Internal packages.** Publish 3.13-compatible builds of internal libraries to Nexus while keeping the 3.11 builds for v2, until v2 is retired.
- **CI.** Add a Python 3.13 job beside the 3.11 job in the DAG repository: install with constraints, compile every DAG file, run a DAG import test (a `DagBag` load with zero import errors) and the unit tests. Keep both jobs until v2 is retired.
- **Base image.** Use the Python 3.13 variant of the official 3.3.2 image (verify the exact tag, for example `apache/airflow:3.3.2-python3.13`), and install extras with the constraints file.

- **Rollback.** v2 stays on Python 3.11 and is never touched, so the Python change adds no rollback risk as long as the DAG release rule above holds.

Phase 0 exit criteria: DAG fixes live on v2 for one business cycle; image built on Python 3.13 and scanned; DAG tests green on Python 3.11 and 3.13; provider, plugin and auth decisions documented; baselines captured.

6. Build the v3 stack (idle, isolated)

Build the entire v3 stack, with all replicas of the scheduling components at zero or all DAGs paused, before the cutover day. Repeat this section per environment.

6.1 Namespace and nodepool

Item	Setting
Namespace	`airflow-v3` (v2 stays in its current namespace)
Nodepool	New dedicated pool with a taint such as `workload=airflow-v3:NoSchedule`; pods use the matching toleration and `nodeSelector`
Sizing	Start at v2's current CPU and memory; add headroom for the extra dag-processor and api-server; adjust after the load test
Quota	Confirm subscription vCPU quota for both pools running at once during the parallel period
Autoscaler	Set min and max for the new pool; avoid scale-to-zero on the pool that runs the scheduler
Pod disruption budgets	One each for api-server, scheduler, dag-processor and workers
Resource quotas and LimitRange	Mirror v2's, then tune

6.2 Networking

- [ ] Network policies allow: workers to api-server (execution API), api-server to the DB, all components to Redis and PostgreSQL, ingress to api-server only.

- [ ] Allow the v3 nodepool subnet in the PostgreSQL firewall or private endpoint rules and in the new Redis firewall.
- [ ] Create an ingress for the test hostname (for example `airflow-v3.<domain>`) with TLS; keep the production hostname on v2 until cutover.
- [ ] Decide the in-cluster execution API URL (the api-server Service DNS name) and set it on all components.
- [ ] Lower the production DNS TTL to 60 seconds at T-2 days (section 10).

6.3 Secrets

Secret	Source	Notes
Fernet key	Same value as v2	Mandatory for decrypting connections and variables
JWT secret	New, random, 32+ bytes	Shared by api-server, scheduler, workers and triggerer
API secret key	New, random	Shared by all api-server replicas
DB connection string	v3 database credentials	Separate role from v2
Broker URL	New Redis	Separate credentials
Celery result backend	v3 database	Never the v2 database
Webserver/SSO client secrets	Existing identity provider app	Add the v3 redirect URI; keep the v2 one
Image pull secret	Same registry	Reference by digest

Store secrets in Key Vault with the CSI secret store driver or in Kubernetes secrets, matching how v2 does it today.

6.4 Redis

- [ ] Provision a new Azure Cache for Redis instance sized for broker use only, with TLS and a private endpoint or VNet rules.
- [ ] Verify the tier and any eviction policy are suitable for a Celery broker (no eviction of queue keys).
- [ ] Record the connection string in the broker secret.

6.5 Database

Detail is in section 7. At build time, only create the roles: a v3 owner role (with `CREATEDB` if you clone on the same server) and a separate v3 application role. The v3 database itself is created by the clone step on cutover day, and again for each rehearsal.

Connection endpoints (PgBouncer). The pooled PgBouncer port (6432) serves all runtime traffic: scheduler, dag-processor, api-server, triggerer, workers and the Celery result backend. The direct port (5432) serves the migration Job, `airflow db clean`, the clone and validation SQL, because transaction pooling can break session-level features and a migration may hold a session-level lock for its whole duration. Use a separate PgBouncer database and user pair for v3, so v2 and v3 each get their own pool (confirm with your DBA how the pool size of 60 is applied).

6.6 Storage (SMB, existing CSI driver)

- [ ] Create a new Azure Files share for v3 with the same tier and quota as v2 (or larger).
- [ ] Create a PV and PVC in `airflow-v3` using the Azure Files CSI driver, access mode `ReadWriteMany`, reclaim policy `Retain`.
- [ ] Use sub-paths: `dags/` and `logs/`. Run a one-time Job to set ownership for UID 50000, GID 0.
- [ ] Mount DAGs on the scheduler, dag-processor, triggerer and workers; mount logs on all components, including the api-server.
- [ ] Keep v2's share and PVC untouched; do not mount it into v3 pods.

SMB mount options that your v2 setup uses today should be copied across; do not tune storage during the upgrade (NFS is Phase 2, section 14).

6.7 Helm release

Use a chart version that supports Airflow 3 (verify against the chart's release notes, appendix D). Review the chart's `values.yaml` for the exact key names; the skeleton below shows intent, not final syntax.

```
executor: CeleryExecutor
images:
  airflow:
    repository: <registry>/airflow
    digest: sha256:<digest>
redis:
  enabled: false              # external Redis
```

```
data:
  metadataSecretName: airflow-v3-db
  brokerUrlSecretName: airflow-v3-broker
fernetKeySecretName: airflow-fernet
# JWT and API secret key references, per the chart's keys
scheduler:   { replicas: 2 }   # start at 0 on build day
apiServer:   { replicas: 2 }
dagProcessor: { enabled: true, replicas: 1 }
triggerer:   { replicas: 1 }
workers:     { replicas: <n> }
nodeSelector: { workload: airflow-v3 }
tolerations:
  - { key: workload, operator: Equal, value: airflow-v3, effect: NoSchedule }
dags:
  persistence:
    enabled: true
    existingClaim: airflow-v3-dags
logs:
  persistence:
    enabled: true
    existingClaim: airflow-v3-logs
config:
  core:
    execution_api_server_url: http://airflow-v3-api-server:8080/execution/
  celery:
    result_backend: db+postgresql://...   # v3 DB
  scheduler:
    catchup_by_default: 'False'
```

Set the auth manager and FAB provider options here if applicable. Install the release with the migration job disabled until section 10, or run it against the empty database in dev.

The metadata secret for the release points to the pooled port (6432) with the v3 database and role. The migration Job gets its own connection string on the direct port (5432).

6.8 Nexus deployment to the second share

- [ ] Extend the release pipeline so one released DAG package version deploys to both the v2 and the v3 share (a Job or deployer pod that pulls from Nexus and unpacks into `dags/releases/<version>`).
- [ ] Keep a `current` link or pointer file per share so a DAG rollback is a redeploy of the previous version.

- [ ] Verify with a checksum compare that both shares hold identical DAG files after each release.
- [ ] Until cutover, the v3 share is test-only; production DAG releases still go live on v2 first.

Build exit criteria: `helm template` and a dry run succeed; all pods in the idle stack schedule on the new pool; the DAG share is populated and checksum-matched to v2; secrets verified; isolation checklist (appendix A) signed.

7. Database strategy: clone, trim, migrate, validate

The v3 database is always a copy of the v2 database that has been migrated. A failed or slow migration costs only the clone, never v2. The schema migration is the longest and least reversible step of the whole upgrade, so it is rehearsed in every environment before it is trusted in production.

7.1 Choose the clone method

Option	How	Pros	Cons	Use when
A. Same server, `CREATE DATABASE ... TEMPLATE`	`CREATE DATABASE airflow_v3 TEMPLATE airflow OWNER airflow_v3_owner;`	Fast for small and medium DBs; no new server	Source must have zero connections; needs `CREATEDB` and membership in the source owner role; needs ~2x storage; shares server IOPS	Dev, UAT, and prod if measured time fits the window
B. Point-in-time restore to a new server	Azure Flexible Server PITR to a new server, then use its database	Full compute and IOPS isolation; v2 server untouched	Slower to create; extra cost; new endpoint and firewall rules	Prod when DB is large or you want hard isolation
C. `pg_dump` and `pg_restore`	Logical dump of v2 DB restored into a new database or server	Works across servers and versions	Slowest	Fallback only

Decide the method in UAT using measured timings. If you choose A for production, confirm the server has enough free storage and spare IOPS for the clone plus the migration.

7.2 Pre-checks (run in every environment)

- [ ] Size of v2 DB: `SELECT pg_size_pretty(pg_database_size('airflow'));`
- [ ] Free storage on the server is at least 2.5x the DB size.
- [ ] PostgreSQL parameters allow the migration's connections and locks; no long-running transactions on the source.
- [ ] Backup retention on the server is 14 days or more; point-in-time restore verified once in dev.
- [ ] Roles created: `airflow_v3_owner` (owns the v3 DB, has `CREATEDB` if using option A) and the v3 application role.
- [ ] Largest tables known (`dag_run`, `task_instance`, `log`, `xcom`, `rendered_task_instance_fields`, `job`); record row counts.
- [ ] Record the server SKU (vCores, memory), the PgBouncer pool mode, `max_client_conn` and idle timeout, and v2's peak connection count from Azure Monitor.
- [ ] Confirm the direct port (5432) and the pooled port (6432) are both reachable from the v3 namespace, and that v3 has its own PgBouncer database and user pair.
- [ ] Set the v3 pool size: start at 60, and raise to 80 to 100 only if the UAT load test shows waiting clients and the SKU has headroom.

7.3 Backups before the real run

1. Trigger an on-demand backup of the v2 Flexible Server (`az postgres flexible-server backup create`) and record its name and time.
2. Take a logical dump of the v2 database to storage outside the cluster: `pg_dump -Fc -d airflow -f airflow_v2_<timestamp>.dump`. Verify it is readable with `pg_restore --list`.
3. Store the backup name, dump file name, checksum and the v2 schema version (`SELECT * FROM alembic_version;`) in the change record.

These are belt and braces. Rollback does not need them because the v2 database is never modified.

7.4 Create the clone

Run only when no process is connected to the source database, and use the direct port (5432), not PgBouncer. On cutover day this is true once all v2 components are scaled to zero (section 10), but PgBouncer keeps idle server connections open after clients leave, so terminate those leftovers before cloning. If your PgBouncer keeps a minimum pool size, ask the DBA how to disable the v2 database in PgBouncer during the clone.

```
-- Direct port (5432). Count must reach zero before cloning.
SELECT count(*) FROM pg_stat_activity WHERE datname = 'airflow' AND pid <>
pg_backend_pid();

-- Leftover idle PgBouncer backends. Run only after v2 is fully stopped.
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE datname = 'airflow' AND pid <> pg_backend_pid();

-- Re-run the count until it returns zero, then clone.
CREATE DATABASE airflow_v3 TEMPLATE airflow OWNER airflow_v3_owner;
```

For rehearsals, name clones by date and attempt, such as `airflow_v3_rehearsal_20261101_1`, and drop them when finished. Never reuse a previously migrated database as the target.

7.5 Trim history on the clone (before migration)

Smaller tables migrate faster. Run the cleanup with the v2 image, because at this point the clone still has the v2 schema. Point it only at the clone.

```
export AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=<clone connection string>
airflow db clean --clean-before-timestamp "<now minus 90 days>" --skip-
archive --yes
```

Run the cleanup on the direct port (5432) of the clone, not through PgBouncer.

- Choose the retention period with the business. Compliance retention requirements apply to the v2 database, which you keep untouched.
- `--skip-archive` avoids creating archive tables that would slow the migration; the full history stays in v2.
- Confirm in rehearsal that the most recent scheduled run per DAG survives the cleanup, because the scheduler uses it to compute the next run.

- If the cleanup takes too long inside the window, do it in a rehearsal clone to measure, then consider running routine housekeeping on v2 a few days earlier as normal operations after taking a backup.

7.6 Migrate

Run the migration as a Kubernetes Job in the v3 namespace with the v3 image, never from a laptop: a dropped connection mid-migration is a known cause of a half-applied schema (upgrade guide).

```
export AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=<clone connection string>
airflow db migrate
airflow db check-migrations --migration-wait-timeout 60
```

Run the migration on the direct port (5432), not through PgBouncer: transaction pooling can break session-level features, and a migration may hold a session-level lock for its whole duration. After the migration, switch the release to the pooled port (6432) for runtime traffic.

- Record start time, end time and the full log.
- If `airflow db migrate` fails, stop. Drop the clone, fix the cause, and repeat from the clone step. Do not try to repair a half-migrated clone in production.
- **Direct versus two-step.** The upgrade guide recommends going via the latest 2.x before Airflow 3. Direct 2.10.5 to 3.3.2 is the chosen path. Prove it in dev first: migrate a clone directly, run the full validation list, and run DAG regression. Fall back to a hop through the latest 2.11.x on the clone only if the migration fails, the 3.3.2 documentation names a minimum starting version that 2.10.5 does not meet, or validation shows data problems. Any hop runs on the clone only and adds its measured time to the window.

7.7 Validate the migrated database

- [ ] Schema version matches the head revision of a fresh `airflow db migrate` run on an empty database in dev.
- [ ] Row counts for `dag`, `dag_run`, `task_instance`, `connection`, `variable`, `slot_pool` and the FAB user and role tables equal the post-clean expectations.
- [ ] No task instances in `running`, `queued`, `scheduled`, `up_for_retry` or `deferred` states.
- [ ] Connections decrypt: `airflow connections get <id>` for three sample connections; `airflow connections test <id>` for connections to stable systems.

- [ ] Variables decrypt: `airflow variables get <key>` for sample keys, including one marked as secret.
- [ ] Pools, DAG paused states and last-run timestamps match the pre-migration export.
- [ ] Save a CSV of `dag_id, is_paused` taken before the migration (used in the controlled start and in the rollback).

7.8 Timing record (fill in per rehearsal)

Step	Dev	UAT	Prod rehearsal	Prod cutover
DB size before				
Clone				
Trim				
Migrate				
Validate				
Total				

Size the production window as the prod rehearsal total times 1.5, plus drain time, plus 2 hours of validation, plus a rollback reserve (section 4).

8. Test plan

Testing runs in three layers: platform tests in dev, DAG and integration tests in UAT, and a dress rehearsal on a fresh clone of production data. A test passes only against the migrated clone, never against an empty database.

#	Area	Test	Layer	Pass criterion
1	Platform	All pods healthy; probes pass (api-server health, scheduler, dag-processor and triggerer job checks)	Dev, UAT, rehearsal	No restarts in 24h
2	Platform	Restart or evict each component during running tasks	Dev, UAT	Tasks complete or retry; no lost runs

#	Area	Test	Layer	Pass criterion
3	Platform	Node drain of the v3 pool	UAT	Pods reschedule; PDBs honored
4	Database	Clone, trim, migrate, validate (section 7), timed	All	Matches validation list; timing recorded
5	Database	Failed migration handling: drop clone and redo	Dev	Redo succeeds; v2 unchanged
6	DAG import	Zero import errors across all DAGs	All	Zero errors; parse time within agreed range of v2 baseline
7	DAG regression	Run every tier-0 DAG end to end against sandbox targets	UAT, rehearsal	Same outputs and status as v2
8	DAG regression	Representative tier-1 and tier-2 DAGs, covering each operator, sensor, deferrable and dynamic mapping pattern in use	UAT	Same outcomes as v2
9	Scheduling	Schedules produce expected run times; cron, timetable, catchup and `start_date` semantics checked on sample DAGs	UAT	Run times match the owner's expectation
10	Scheduling	Pool, priority weight, concurrency and `max_active_runs` limits behave as on v2	UAT	Limits respected
11	Connections	Decrypt and connect to each system class (databases, Azure services, SFTP, HTTP)	UAT	All connection tests pass
12	Secrets	Secrets backend lookups work from workers and the dag-processor	UAT	Values resolve

#	Area	Test	Layer	Pass criterion
13	Auth	Login, roles and permissions mapped from v2; least-privilege review	UAT	Role mapping signed by security
14	API	Every external caller works against `/api/v2` with the new token flow	UAT	Callers pass in test
15	Logs	Task logs visible in UI for success, failure, retry and running tasks	All	Logs readable within seconds
16	Plugins	Custom plugins and UI extensions load and function	UAT	No errors in logs
17	Performance	Parse time, scheduler loop time, task start latency, worker utilization vs v2 baseline	UAT, rehearsal	No worse than 10% against baseline, or explained
18	Performance	Load test with peak-hour DAG count and concurrency	UAT, rehearsal	No queue starvation; DB CPU within limits; PgBouncer waiting clients and wait time stay low
19	Observability	Metrics, logs and alert rules updated for api-server and dag-processor	UAT	Alerts fire in a test
20	Security	Network policies enforced; TLS; JWT secret rotation test; image scan clean	UAT	Security sign-off
21	Storage	Shares mount on all pods; ownership correct; DAG release deploy to both shares	All	Checksums match
22	Cutover	Full cutover rehearsal with timings (section 10)	Dev, UAT, rehearsal	Within the planned window

#	Area	Test	Layer	Pass criterion
23	Rollback	Full rollback drill with timings (section 12)	Dev, UAT	Under 30 minutes; v2 fully working
24	Reconciliation	Rollback after some v3 runs; missed-run handling for sample tier-0 DAGs	UAT	No duplicates, no gaps
25	Python	DAG import, unit tests and dependency install on Python 3.13 (v3) and Python 3.11 (v2) from the same DAG release	CI, dev, UAT	Zero failures on both; no removed-module imports

Test data and safety.

- Point test DAGs at sandbox or masked targets. Production-like clones carry real connection details, so pause all DAGs on the clone and unpause only the DAGs you are testing, with side-effecting tasks redirected.
- Never start a v3 scheduler on a production clone with DAGs unpaused and production credentials reachable.

Exit criteria for each layer.

- Dev: tests 1 to 6, 21 to 23 pass.
- UAT: all 24 tests pass or have an accepted, documented exception signed by the change owner.
- Prod rehearsal: tests 1, 4, 6, 17, 21 and 22 pass on a fresh production clone; timings fit the window.

9. Environment rollout sequence

Dev proves the mechanics, UAT proves the DAG estate and the timings, and production repeats the exact runbook that UAT passed. No environment starts until the previous one has met its exit criteria.

Environment	Entry criteria	What happens	Exit criteria
Dev	Phase 0 complete; image built	Build v3 stack (section 6); clone, migrate and validate (section 7); run tests 1-6 and 21-23; fix the runbook	Dev cutover and rollback drills done; runbook v1 corrected; direct-versus-two-step migration decision made
UAT	Dev exit met; DAG fixes live on UAT v2 for one business cycle	Same build; clone of UAT DB; full test matrix (section 8); timed cutover and rollback drills; load test; security and role review	All tests pass; timings recorded; production window sized; business sign-off on tier-0 DAGs
Prod, build and rehearse	UAT exit met; production change approved; DAG fixes live on prod v2	Build idle v3 stack; fresh clone of the production DB; migrate and validate on the clone with all DAGs paused; run tests 1, 4, 6, 17, 21, 22	Rehearsal timings fit the window; go/no-go 1 passed; the rehearsal clone is dropped
Prod, cutover	Go/no-go 2 passed at T-1 day; freeze in effect	Run section 10	Go/no-go 3 at T+4h and T+24h
Prod, stabilization	Cutover complete	v2 on standby for 14 days (section 13)	No Sev1/Sev2; sign-off to retire v2

Rules for every environment.

- The environment's v2 stays running until its own cutover starts. v3 for that environment exists only as an idle stack plus rehearsal clones until then.
- Use the same hostnames pattern, secrets pattern and Helm values structure in all three environments, differing only by environment values.
- Fix defects in the runbook text immediately after each drill, and re-run the corrected steps in the next environment.

- Do not start the next environment's cutover in the same week as a failed or rolled-back one.

Go/no-go gates.

Gate	When	Needs
1	After the production rehearsal	Timings fit; tests pass; rollback drill from UAT is current; v3 idle stack healthy
2	T-1 day	Backups verified; freeze in effect; DAG release identical on both shares; DNS TTL lowered; approvals and comms sent; on-call staffed
3	T+4h and T+24h	Section 11 criteria met; otherwise follow section 12

10. Production cutover runbook

The cutover drains v2, clones and migrates its database, starts v3 with all DAGs paused, switches traffic, then unpauses DAGs in tiers. Every step has an owner, a check and an abort path. Offsets are relative to T0, the start of the window; replace the indicative times with measured values from the rehearsal.

10.1 Before the window

When	Action	Owner
T-14d	Freeze infrastructure changes on both stacks; v3 idle stack healthy; prod rehearsal scheduled	Platform
T-7d	Prod rehearsal on a fresh clone; record timings; go/no-go 1	Platform, DBA
T-3d	Confirm approvals, window, comms list, on-call rota, bridge details	Change owner
T-2d	Lower production DNS TTL to 60s; add v3 SSO redirect URI to the identity provider; confirm the API caller owners are ready	Cloud, Security
T-2d	DAG release freeze starts: only releases that go to both shares and are compatible with v2 and v3	Release

When	Action	Owner
T-1d	Confirm checksums match between the v2 and v3 DAG shares; confirm v2 backups healthy; confirm quota; go/no-go 2	Platform
T-1h	Last health check of v2 and the idle v3 stack; confirm tier-0 DAG owners on the bridge	Change owner

10.2 Cutover day

Phase A: Start and snapshot v2 state (T+0:00)

1. Open the bridge, announce the start, and confirm the roster and the rollback authority.
2. Record the v2 state for the change record: `helm list -n <v2-ns>`, `kubectl get pods -n <v2-ns> -o wide`, image digests, and `helm get values <v2-release> -n <v2-ns> -o yaml > v2-values-T0.yaml`.
3. Export, read-only from the v2 database, the DAG paused states: `\copy (SELECT dag_id, is_paused FROM dag ORDER BY dag_id) TO 'dag_paused_T0.csv' CSV HEADER`. Also export pool definitions, the connection IDs and the variable keys.
4. Record the v2 schema version and the row counts of `dag`, `dag_run`, `task_instance`, `connection`, `variable`, `slot_pool`.

Phase B: Drain and stop v2 (T+0:15)

1. Scale the v2 scheduler to zero so no new runs start: `kubectl scale deploy/<v2-release>-scheduler -n <v2-ns> --replicas=0` (use the workload kind your release uses).
2. Wait for running work to finish. Poll until the result is zero or the drain limit is reached:

```
SELECT state, count(*) FROM task_instance
WHERE state IN
('running','queued','scheduled','up_for_retry','restarting','deferred')
GROUP BY state;
```

3. Drain limit: agree it in advance (for example 30 to 45 minutes). At the limit, tier-0 owners choose per task whether to wait or stop it. Record every task stopped; v3 reruns them after the migration.
4. Scale to zero, in this order: triggerer, workers, webserver, and any flower or exporter pods. Leave the Helm release installed.

5. Confirm no v2 pods remain: `kubectl get pods -n <v2-ns>`.
6. Confirm the v2 database has no connections:

   ```
   SELECT usename, application_name, client_addr, state FROM pg_stat_activity
   WHERE datname = 'airflow' AND pid <> pg_backend_pid();
   ```

   Run this on the direct port (5432). Monitoring exporters, BI tools and manual sessions must be stopped. Idle backends left behind by PgBouncer are expected, because it holds server connections open after clients leave until its idle timeout: terminate them with pg_terminate_backend (safe, since v2 is stopped), then re-run the query until it returns zero.
7. Confirm the v2 Redis queues are empty (for example `LLEN` on each Celery queue name your config uses).
8. Update the status page or announcement: Airflow is paused.

Checkpoint 1. v2 is fully stopped, the database has zero connections, no tasks are in flight. If this takes longer than the drain limit plus 15 minutes, the change owner decides: extend, or abort and restart v2 (abort path A below).

Phase C: Protect, clone, migrate, validate (T+0:45)

1. Trigger the on-demand backup of the v2 server (section 7.3) and record its name. Take the logical dump if the rehearsal showed it fits the window.
2. Create the clone (section 7.4): `CREATE DATABASE airflow_v3 TEMPLATE airflow OWNER airflow_v3_owner;`, or restore to the new server if option B was chosen.
3. Trim history on the clone with the v2 image (section 7.5).
4. If tasks were stopped at the drain limit, fix their states on the clone only (for example set them to `failed` so the scheduler retries or you rerun them). Never change task states in the v2 database.
5. Run the migration Job with the v3 image (section 7.6). Watch the log; do not interrupt it.
6. Run the validation list (section 7.7).
7. Prepare the controlled start: on the v3 database, set every DAG to paused (`UPDATE dag SET is_paused = true;`), after confirming `dag_paused_T0.csv` is stored safely.

Checkpoint 2. Migration finished, validation passed, total time at most 1.5x the rehearsal. If the migration failed or ran past the limit, take abort path B.

Phase D: Start v3 (T+2:00)

1. Confirm the v3 DAG share holds the intended release and its checksums match the v2 share.
2. Point the v3 release at the migrated database (update the DB secret, then `helm upgrade --install airflow-v3 ... -n airflow-v3`), with scheduler, triggerer and workers at zero replicas.
3. Start the api-server and the dag-processor. Check: api-server health endpoint is healthy; login works through SSO; the DAG list matches the expected count; import errors equal zero.
4. Start workers and the triggerer, then the scheduler last. Check scheduler, dag-processor and triggerer job checks, and that workers register with the broker.
5. Run the smoke DAG: unpause it (schedule is none) and trigger it. It must touch one connection, one variable, one pool, push and pull an XCom, write a log, and fire a callback.
6. Check the logs view in the UI for the smoke run, and call the REST API with a token to read the DAG list.

Checkpoint 3. All smoke checks pass and no pod has restarted. If not, fix within 20 minutes or begin the rollback in section 12 (the DB switch has not been announced yet, so this is still a clean rollback).

Phase E: Switch traffic (T+2:30)

1. Change the production hostname (DNS record or ingress) to the v3 api-server. Keep the v2 ingress object in place but pointing at the scaled-down service.
2. Verify TLS, SSO login and the UI from outside the cluster.
3. Hand the new token flow and `/api/v2` URL to every external caller and check each caller works, including CI triggers and monitoring probes.

Phase F: Controlled unpause (T+3:00)

1. Read `dag_paused_T0.csv`. Only DAGs that were unpaused at T0 are candidates; never unpause a DAG that was paused at T0.
2. Unpause tier-0 DAGs in small batches (`airflow dags unpause <dag_id>` or the API). For each batch, watch the first runs in the UI and have the DAG owner confirm results.
3. If no problems appear for 30 minutes, unpause tier-1, then tier-2.
4. Watch for a burst of catchup runs: queued run counts, pool slot use, worker queue length. Pause a DAG immediately if it starts unexpected backfill.
5. Record which DAGs created missed-interval runs and confirm each one was expected.

Phase G: Hypercare

1. Announce completion. Keep the bridge open for the first 2 hours, then use the on-call rota.
2. Review the dashboard at T+1h, T+4h and T+24h against the criteria in section 11; go/no-go 3 at T+4h and T+24h.
3. Keep the DAG freeze in force. Post-cutover releases go to both shares and must remain v2-compatible until v2 is retired.
4. Record actual timings per phase in the change record and update the timing table in section 7.8.

10.3 Abort paths before the point of no return

Path	Condition	Action	Data impact
A	v2 will not drain within the allowed time	Scale v2 components back up (workers, triggerer, webserver, then scheduler); verify; resume	None
B	Clone or migration fails, or runs past the limit	Drop the clone; scale v2 back up as in A; reschedule	None; v2 DB untouched
C	v3 fails checkpoint 3	Scale v3 to zero; scale v2 back up as in A; the traffic switch has not happened	None
D	Anything after traffic switch	Full rollback, section 12	v3-window activity must be reconciled

The real point of no return is not a moment but the end of the rollback window (section 12.1): after it, v2 is fixed forward rather than restarted.

11. Post-cutover validation and go/no-go criteria

Go/no-go 3 is decided at T+4h and T+24h against measured thresholds, so the rollback decision is not made on feelings. Set the numeric thresholds from your v2 baselines (section 5.1) before the window; the values below are starting proposals.

Signal	Healthy	Rollback trigger (Sev1)	Investigate (Sev2)
Scheduler heartbeat	Fresh every few seconds; no gaps	No heartbeat for 5 minutes, not recovering after one pod restart	Intermittent gaps
DAG import errors	Zero	Any import error on a tier-0 DAG that cannot be fixed within 30 minutes	Errors on tier-1 or tier-2 DAGs
Task success rate vs v2 baseline	Within 2 percentage points	More than 10 points worse, not explained by data or external systems	2 to 10 points worse
Tier-0 DAG runs	On time and correct	Any missed, duplicated or wrong-output tier-0 run caused by the platform	Delay under the DAG's SLA
Queue latency (queued to running)	Within 20% of baseline	Tasks wait more than 15 minutes with free worker capacity	20 to 100% above baseline
API server errors	Rare 5xx	Sustained 5xx above 5% for 10 minutes	Isolated 5xx spikes
Worker to api-server calls	Stable	Task failures from execution API errors or token failures	Occasional retries
DB CPU, connections, IOPS, PgBouncer waits	Below 70% of capacity	Saturated for more than 10 minutes	70 to 90%
DAG parse time	Within 20% of baseline	Parse loop cannot finish and DAG updates do not appear	20 to 100% above baseline
Auth and SSO	Users and API clients sign in	Widespread login failure or a permission model that exposes data	Individual role gaps

Signal	Healthy	Rollback trigger (Sev1)	Investigate (Sev2)
Logs	Task logs readable in the UI	Logs unavailable for running or failed tasks across DAGs	Slow log loading
Pod stability	No restarts	Crash loops on scheduler, api-server or workers	Single restarts

Checks at each gate.

- [ ] T+1h: smoke DAG green; first tier-0 runs correct; no pod restarts; no unexpected catchup burst.
- [ ] T+4h: all tier-0 DAGs that were due have run correctly; owners confirm outputs; metrics inside the healthy column.
- [ ] T+24h: a full day of schedules, including any overnight batch, ran; error budget reviewed; logs and alerts reviewed.
- [ ] T+7d: a weekly or month-boundary schedule (if any) has run; capacity review done.

Decision rule.

- Any Sev1 signal at any time: the rollback authority starts section 12 within 15 minutes unless a fix is already running and expected within 30 minutes.
- Sev2 signals: raise an incident, fix forward, and review again in 4 hours.
- Sign-off to leave hypercare needs 7 days without Sev1 and 14 days without unresolved Sev2 before v2 is retired (section 13).

12. Rollback plan

Rollback is: stop v3, start v2 against its untouched database with all DAGs paused, reconcile what v3 did, then unpause in tiers and switch traffic back. The target is v2 serving again within 30 minutes of the rollback decision, with a reconciliation plan for the v3 window agreed within 2 hours.

12.1 Principles, window and objectives

Item	Definition
Rollback window	14 days after cutover, while v2 and its database are retained untouched
Time to restore service (RTO)	30 minutes from the decision to v2 serving traffic, proven in the UAT drill
Data loss objective (RPO)	v2 holds state as of the cutover moment; run history created in v3 is exported, not merged back
Decision authority	The change owner, with a named deputy; the on-call engineer can pause DAGs and scale v3 to zero without approval in a Sev1
Single-writer rule	Only one stack may run schedulers at any time; v3 must be fully stopped before v2 starts
After the window	Fix forward. Restoring v2 later is a restore from backup, which is a different and slower procedure (section 12.8)

12.2 What must stay true so rollback works

Requirement	How it is protected
v2 database is unmodified	Clone-only operations; v2 role has no v3 jobs; checked at cutover
v2 Helm release, images and values still available	Release not uninstalled; image digests recorded; `v2-values-T0.yaml` saved
v2 DAG share has a v2-compatible release	DAG freeze; every release goes to both shares and stays v2-compatible; `current` pointer per share
v2 Redis unchanged and queues empty	v3 uses its own Redis; queues verified empty at drain
v2 secrets unchanged	Separate v3 secrets; no shared secret rotation during the window

Requirement	How it is protected
v2 nodepool and quota still available	Do not shrink or upgrade the v2 pool during the window; keep the AKS version stable
Original DNS and ingress definition known	Previous record values saved; TTL kept at 60s until the window ends
Connection and variable changes in v3 are tracked	Freeze admin changes in v3, or export diffs daily
Paused-state export from T0	`dag_paused_T0.csv` stored in the change record

12.3 Rollback procedure

Stage 1: Stop the bleeding (target 10 minutes)

1. The rollback authority declares rollback on the bridge, names the time (call it TR), and freezes all other changes.
2. Stop new runs on v3: scale the v3 scheduler to zero (or pause all DAGs through the API if the scheduler is unreachable).
3. Record what is in flight on v3 (`task_instance` rows in running, queued, deferred or retry states) and let short tasks finish if they are safe to finish. Stop the rest after tier-0 owners confirm; write each stopped task to the rollback log.
4. Export the v3-window activity for reconciliation (section 12.4): runs and task outcomes since cutover, plus any connection, variable and pool changes. Store the export outside the cluster.

Stage 2: Stop v3 completely (target 5 minutes)

1. Scale to zero: triggerer, workers, dag-processor, api-server. Leave the Helm release, the v3 database and the v3 share untouched for the post-mortem.
2. Confirm no v3 pods run: `kubectl get pods -n airflow-v3`.
3. Confirm no v3 process holds connections to the v2 database (there should be none, since v3 uses its own database): query `pg_stat_activity` on the v2 database and check it is quiet.
4. Do not delete v3 resources. Do not run `helm uninstall`.

Stage 3: Bring v2 back (target 10 minutes)

1. Check the v2 database is as left at T0: schema version as recorded, row counts equal the T0 export, no sessions connected.
2. Check the v2 DAG share: the `current` release is the last released version, and checksums match the v3 share (a compatible release).
3. Pause every DAG in v2 before the scheduler can act, so catchup logic does not create runs for intervals that v3 already ran. Using a one-off Job with the v2 image or a read-write session on the v2 database: `UPDATE dag SET is_paused = true;`. The saved CSV records the original states.
4. Scale up v2 in this order: webserver, triggerer, workers, then the scheduler last. Check scheduler heartbeat, worker registration and the webserver health.
5. The v2 scheduler will find task instances left `running` from the pre-cutover drain, if any were forced. Let its zombie detection mark them, or mark them as failed by policy, then review them in the rollback log.

Stage 4: Verify and switch traffic (target 5 minutes)

1. Run the v2 smoke DAG (the same smoke DAG deployed on both stacks): connection, variable, pool, XCom, log, callback.
2. Confirm import errors equal zero and the DAG count matches the T0 export.
3. Change the production hostname back to the v2 webserver (restore the previous DNS or ingress target; TTL is 60s). Confirm TLS and SSO on the v2 hostname still work.
4. Tell API callers to switch back to `/api/v1` and the v2 authentication method; verify each caller.

Stage 5: Controlled restart of scheduling (reconciliation window)

1. Apply the reconciliation decisions (section 12.4), then unpause DAGs by tier using the T0 CSV, tier-0 first.
2. After each batch, watch the first runs and confirm with the DAG owners.
3. Announce that service is restored and that a post-mortem follows.

12.4 Reconciling the v3 window

v2's database does not know about runs that v3 performed. If v2 simply starts its scheduler, it will recreate runs and may repeat work. Handle this explicitly per DAG.

DAG type	Risk when v2 restarts	Action
Idempotent pipelines (overwrite or upsert)	Low; rerun is safe	Let v2 catch up per its `catchup` setting
`catchup=False` DAGs	One extra run for the latest interval	Accept if idempotent; otherwise mark that interval complete before unpausing
`catchup=True` DAGs	v2 creates every interval since its last run, including ones v3 completed	Decide with the owner: allow reruns if idempotent; otherwise mark those intervals complete in v2 before unpausing (validate this technique in the UAT drill)
Non-idempotent, side-effecting DAGs (emails, payments, external writes, appends)	Duplicate side effects	Tier-0 owner manually decides the interval list; keep these paused until a safe plan exists
Event or asset-triggered DAGs	Missed or duplicated triggers	Replay events deliberately from the source
Sensors waiting on external state	Wait on already-consumed signals	Re-arm or skip per owner

Data to export from v3 at the start of Stage 1.

- DAG runs and task instance outcomes since cutover: `dag_id, run_id, logical date or run_after, start, end, state`.
- Any connection, variable, pool or role changes made in v3; apply them to v2 only if still needed.
- XCom or asset events that downstream systems consumed (list only, for owners to check).
- External side effects reported by owners (files written, messages sent, tables loaded).

Rule. Business owners, not platform engineers, approve how each tier-0 DAG resumes.

12.5 Verification after rollback

- [ ] v2 UI and API reachable at the production hostname; SSO works.
- [ ] Scheduler heartbeat, worker registration and triggerer healthy.
- [ ] Import errors zero; DAG count equals T0 export.
- [ ] Smoke DAG green; tier-0 DAGs resumed per reconciliation decisions with first runs confirmed.
- [ ] No pods running in `airflow-v3`; v3 database and share preserved and labeled for the post-mortem.
- [ ] API callers pointed back and verified.
- [ ] Rollback log complete: times, people, decisions, tasks stopped, DAGs re-run or skipped.

12.6 Rollback drills (mandatory)

Drill	Where	Pass criterion
Full cutover then immediate rollback	Dev, after the first successful cutover	Procedure works as written; defects fixed
Cutover, 2 hours of v3 runs on sample DAGs, then rollback with reconciliation	UAT	Restore of service within 30 minutes; no duplicates or gaps on sample tier-0 DAGs
Timed repeat with a different engineer reading the runbook	UAT	Same result; no tribal knowledge needed
Abort paths A, B and C (section 10.3)	Dev	Each returns to v2 cleanly

12.7 After a rollback

- [ ] Communicate status and next steps to stakeholders within 2 hours.
- [ ] Preserve evidence: v3 logs, pod events, the v3 database and the rollback log.
- [ ] Hold a blameless review within 3 working days; list root causes and fixes.
- [ ] Fix, then re-test in dev and UAT, and repeat the production rehearsal on a fresh clone before re-attempting.
- [ ] Clean up in the right order: drop v3 rehearsal databases only after the review; never delete the v3 database used in the failed cutover until the review closes.

12.8 If v2 itself cannot start, or the window has ended

Situation	Action
v2 pods fail on startup	Check image digest, secrets, DAG share mount and Redis connectivity against the T0 record; compare with `v2-values-T0.yaml`
v2 database appears damaged (not expected)	Restore from the on-demand backup or point-in-time restore to the T0 timestamp, then repoint v2; allow time for restore (measured in the dev drill)
v2 DAG share damaged	Redeploy the last released DAG version from Nexus to the v2 share
After the 14-day window	v3 is the only live stack; fix forward, or restore v2 from backup as an emergency procedure that needs its own change approval

What rollback cannot undo. Actions v3 runs already took in external systems (writes, messages, payments) are not reversed by this procedure; the reconciliation in section 12.4 is how those are accounted for.

13. Stabilization and v2 decommissioning

v2 stays installed but scaled to zero for 14 days after the production cutover, then is retired in stages so nothing irreversible happens early.

Day	Action	Condition
0 to 2	Hypercare with a named rota; daily review of section 11 signals; DAG freeze in force	Gate 3 passed at T+4h and T+24h
3 to 7	Weekly-cycle DAGs observed; capacity and cost review of the new pool; alert tuning	No Sev1
7	Freeze relaxes: DAG releases may go to v3 only if they are v2-compatible, or the team accepts the loss of fast rollback	7 days without Sev1

Day	Action	Condition
14	Decision meeting to retire v2	No open Sev2; owners sign off tier-0 DAGs
14 to 21	Stage 1 retirement: uninstall the v2 Helm release; delete v2 workloads and the v2 Redis instance; keep the v2 database, the v2 share and the final dump	Change approved
21 to 90	Keep a final v2 database backup and DAG share snapshot for audit and last-resort restore	Per retention policy
After 90	Stage 2: delete the v2 database, v2 share and old secrets; remove the old nodepool or shrink it for other uses	Compliance sign-off

Cleanup checklist.

- [ ] Remove v2 hostnames, ingress objects, SSO redirect URIs and DNS entries.
- [ ] Remove the v2 deploy target from the Nexus pipeline; v3 becomes the only target.
- [ ] Drop all rehearsal clone databases.
- [ ] Raise DNS TTL back to the normal value.
- [ ] Update dashboards, alerts, runbooks and on-call documentation to v3 component names.
- [ ] Modernize DAG code with the deferred AIR311/AIR312 recommendations, once v2 compatibility is no longer needed.
- [ ] Capture lessons learned and the final timing record for the platform wiki.

Cost note. Two stacks run in parallel for 14 or more days. Scale the v2 stack to zero from cutover, keep the v2 nodepool at minimum size, and confirm the nodepool autoscaler does not remove the pool in a way you cannot restore quickly.

14. Phase 2 (after stabilization): NFS and Blob logging

Both items are deliberately excluded from the migration so a storage or logging problem cannot be mistaken for an Airflow 3 problem, and so rollback stays simple. Start Phase 2 only after v2 is retired or after a separate change approval, one item at a time, dev first.

Item	Why wait	Preconditions	Test before production
NFS 4.1 premium share for DAGs	Parse speed on SMB may be acceptable; do not change storage type and Airflow version together	Measured v3 parse times on SMB are a problem; premium FileStorage account; private endpoint or service endpoint on the AKS subnet; Nexus deployer able to mount the NFS PVC	Compare parse time and scheduler loop time on SMB and NFS with the same DAG set
Azure Blob remote logging	Remote-logging resolution changed in the 3.x line (legacy fallback goes away in Airflow 4.0), so verify it in isolation	Workload identity or managed identity set up; storage account and container with lifecycle policy; provider version for Azure remote logging verified for 3.3.2	Run failing, retrying and long-running tasks in UAT; check UI log display and retention

See the 3.3.x release notes on remote logging.

14.1 Sequencing and rules for Phase 2

- Phase 2 starts only after the 14-day stabilization window and sign-off (section 13). Each item is its own change, tested in dev, then UAT, then production. Never change both in one window.
- Both items are independent. Start with the one that measurements justify: NFS if v3 DAG parse time or scheduler loop time on SMB misses the targets in section 11; Blob logging if log retention, availability or share I/O is the problem. If both are justified, do NFS first because it affects scheduling performance.
- Every item keeps the old path alive during a soak period, so rollback is a configuration revert and not a data recovery: the SMB DAG share keeps receiving every release, and local logs stay on the share until the Blob path is trusted.
- Entry criteria: v3 baselines captured, change approved, on-call rota staffed, and a tested rollback step written into the change.

14.2 NFS 4.1 premium share for DAGs

Goal. Faster DAG file parsing and metadata operations than SMB, with the same no-rebuild deployment from Nexus. Only the DAG share changes; logs stay on SMB until section 14.3.

Prerequisites.

Requirement	Detail
Storage account	Premium FileStorage account (SSD); NFS 4.1 is Linux only and available only on premium shares, as LRS or ZRS
Size and billing	Provisioned capacity billing with a 100 GiB minimum per share; size for DAG releases plus headroom
Authentication	None: NFS has no key or identity authentication, so access is controlled by the network only
Network	Private endpoint, or a service endpoint on the AKS nodepool subnet with a matching storage account network rule
Secure transfer	The account setting "secure transfer required" must be off for NFS
Security review	Data in transit on the VNet is not encrypted by NFS 4.1; get security sign-off for that
Deployment access	NFS shares are not reachable through the storage REST API or azcopy, so the Nexus deployer must mount the volume

Build steps. Flag names can change between Azure CLI versions; check each command's `--help`.

```
# Premium FileStorage account
az storage account create -n <acct> -g <rg> -l <region> \
  --sku Premium_LRS --kind FileStorage \
  --https-only false --allow-blob-public-access false --default-action Deny

# NFS share (size for releases plus headroom)
az storage share-rm create --storage-account <acct> -g <rg> \
  --name airflow-dags-nfs --quota 100 \
  --enabled-protocols NFS --root-squash NoRootSquash

# Private endpoint (and link privatelink.file.core.windows.net to the VNet)
az network private-endpoint create -n pe-airflow-nfs -g <rg> \
  --vnet-name <vnet> --subnet <pe-subnet> \
  --private-connection-resource-id <storage-account-id> \
  --group-id file --connection-name airflow-nfs
```

Static PV and PVC (Azure Files CSI driver, NFS protocol; the share keeps `Retain` so deleting the claim never deletes the data).

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: airflow-dags-nfs
spec:
  capacity:
    storage: 100Gi
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: azurefile-csi-nfs     # label only, used to bind the
claim
  mountOptions:
    - nconnect=4
    - actimeo=30
  csi:
    driver: file.csi.azure.com
    volumeHandle: <rg>#<acct>#airflow-dags-nfs#   # must be unique per PV
    volumeAttributes:
      resourceGroup: <rg>
      storageAccount: <acct>
      shareName: airflow-dags-nfs
      protocol: nfs
---
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: airflow-v3-dags-nfs
  namespace: airflow-v3
spec:
  accessModes: [ReadWriteMany]
  storageClassName: azurefile-csi-nfs
  volumeName: airflow-dags-nfs
  resources:
    requests:
      storage: 100Gi
```

One-time ownership Job. The Airflow image runs as UID 50000, GID 0. Run a Job as root that creates `releases/` and sets ownership and group write permission. If you see permission errors later, check the share's root-squash setting first.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: nfs-dags-chown
  namespace: airflow-v3
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: chown
          image: <registry>/busybox:<tag>
          securityContext:
            runAsUser: 0
          command: ["sh", "-c", "mkdir -p /dags/releases && chown -R 50000:0
/dags && chmod -R g+rwX /dags"]
          volumeMounts:
            - name: dags
              mountPath: /dags
      volumes:
        - name: dags
          persistentVolumeClaim:
            claimName: airflow-v3-dags-nfs
```

Nexus deployer for NFS. Extend the release pipeline with a Job (or deployer pod) that mounts the claim, pulls the released DAG package from Nexus, verifies its checksum and unpacks it into `releases/<version>`. It also writes the same version to the SMB share until the soak ends.

```
set -euo pipefail
VERSION="$1"; TARGET="/dags/releases/$VERSION"
mkdir -p "$TARGET"
curl -fsSL -u "$NEXUS_USER:$NEXUS_PASS"
"$NEXUS_URL/<path>/dags-$VERSION.tar.gz" -o /tmp/dags.tgz
echo "$EXPECTED_SHA256  /tmp/dags.tgz" | sha256sum -c -
tar -xzf /tmp/dags.tgz -C "$TARGET"
chown -R 50000:0 "$TARGET"
ln -sfn "releases/$VERSION" /dags/current.tmp && mv -Tf /dags/current.tmp
/dags/current   # atomic switch
```

Symlinks work natively on NFS. Before relying on the `current` link, test in UAT how the dag-processor reacts when the link target changes (file paths change with each release). If it causes unwanted DAG re-parsing or version churn, unpack in place instead.

Rehearsal (UAT, then a production-sized test).

1. Populate the NFS share with the same DAG release as SMB through the deployer; compare checksums.
2. Run the same DAG set against SMB and against NFS in separate test namespaces.
3. Compare DAG parse time, scheduler loop time and the delay between publishing a new DAG file and seeing it in the UI (the `actimeo` setting trades freshness for fewer metadata calls).
4. Adopt NFS only if the improvement meets the target agreed from the v3 SMB baseline.

Production cutover (about 30 to 60 minutes; measure it in UAT).

1. Announce the window. Deploy the current DAG release to the NFS share and confirm checksums match the SMB share.
2. Scale the scheduler to zero and drain running tasks using the same drain limit as section 10.
3. Change the release to the new claim (`dags.persistence.existingClaim` in the chart values) and roll the dag-processor, triggerer and workers, then the scheduler last.
4. Verify the mount on each pod (`df -h` shows the NFS endpoint), import errors are zero, the DAG count is unchanged, and the smoke DAG passes.

5. Watch the first runs of tier-0 DAGs and compare parse time with the rehearsal.
6. Keep the pipeline deploying every release to both shares during a 7-day soak.

Rollback. Point the release back to the SMB claim and restart in the same order. This takes about 15 minutes, and the DAG content is identical because both shares received every release.

Watch-outs.

- NFS shares cannot be browsed or written through azcopy or the portal file browser; all writes go through a pod.
- Check whether share-level snapshots and backup are supported for NFS shares in your region. If not, DAG recovery is a redeploy from Nexus, which your process already supports.
- Confirm private endpoint DNS resolves from the nodepool, and that the nodepool subnet is allowed on the storage account.
- Zone and region support differs between LRS and ZRS premium shares; confirm before choosing.

14.3 Azure Blob remote logging

Goal. Durable, retention-managed task logs in Blob storage, so logs survive pod and share problems and the log share stops being a scaling concern.

How it should behave (verify against the pinned provider docs). Workers write task logs locally while the task runs and upload them when it finishes. The api-server shows running-task logs from the worker and finished-task logs from Blob. Release notes for the 3.3 line say remote-log resolution was decoupled from the old logging configuration module, and provider handlers now read their settings through a `from_config` hook (release notes). Read the Azure provider's documentation for the exact version pinned by the 3.3.2 constraints file before configuring.

Prerequisites.

Requirement	Detail
Storage account	General-purpose v2 account and a container such as `airflow-logs`; redundancy per your policy
Network	Private endpoint or service endpoint from the AKS subnet; private DNS for the blob endpoint

Requirement	Detail
Protection	Blob soft delete enabled; no public access
Retention	A lifecycle policy, for example move to cool after 30 days and delete after your retention period
Identity	AKS OIDC issuer and workload identity enabled; user-assigned managed identities with federated credentials; no storage account keys in Airflow
Provider	`apache-airflow-providers-microsoft-azure` at the version pinned by the 3.3.2 constraints
Cost	Estimate capacity and transaction costs from current log volume per day

Identity setup (least privilege). Use a writer identity for the components that produce logs (workers, triggerer, and the dag-processor if you ship its logs) and a read-only identity for the api-server.

```
az identity create -n id-airflow-logs-writer -g <rg>
az identity federated-credential create --name airflow-v3-worker \
  --identity-name id-airflow-logs-writer -g <rg> \
  --issuer <aks-oidc-issuer-url> \
  --subject system:serviceaccount:airflow-v3:<worker-service-account> \
  --audiences api://AzureADTokenExchange
az role assignment create --assignee <writer-client-id> \
  --role "Storage Blob Data Contributor" \
  --scope <storage-account-id>/blobServices/default/containers/airflow-logs
# Repeat the federated credential per component service account.
# Reader identity for the api-server: role "Storage Blob Data Reader".
```

Label the pods with `azure.workload.identity/use: "true"` and annotate each service account with the identity's client ID, through the chart's service account values.

Airflow settings (typical names; confirm against the pinned provider version).

```
AIRFLOW__LOGGING__REMOTE_LOGGING=True
AIRFLOW__LOGGING__REMOTE_LOG_CONN_ID=azure_blob_logs     # wasb connection
using workload identity, no keys
AIRFLOW__LOGGING__REMOTE_BASE_LOG_FOLDER=wasb-airflow-logs
AIRFLOW__AZURE_REMOTE_LOGGING__REMOTE_WASB_LOG_CONTAINER=airflow-logs
AIRFLOW__LOGGING__DELETE_LOCAL_LOGS=False                # keep local copies
during the soak
```

Test matrix (dev, then UAT).

Case	Expected
Successful task	Log appears in Blob and in the UI shortly after the task ends
Failed task and each retry attempt	Separate logs per attempt, all readable
Skipped and upstream-failed tasks	No errors from the log handler
Long-running task	Log visible while running, then complete in Blob
Deferrable task and trigger logs	Both parts readable
Worker pod killed mid-task	Task fails or retries as before; the partial log is understood and acceptable
Very large log	Uploads complete; UI loads without timeout
api-server restart	Logs still readable
Role removed from the writer identity	Tasks still succeed; clear errors in logs; alert fires
Private endpoint DNS	Resolves from all pods that write or read logs
Lifecycle rule	Test with a short retention on a test container; deletion and cool tier work

Production rollout.

1. Create the storage account, container, identities, role assignments and lifecycle policy in advance.
2. In a short window, add the settings to the release and roll the workers, triggerer and api-server. Run the smoke DAG and a tier-0 DAG.
3. Keep `delete_local_logs` false for 14 to 30 days; local logs remain the fallback.

4. After the soak, decide whether to turn on local log deletion and shrink the log share.

Historical logs. Logs from before the switch stay on the share. Options: leave the share mounted and readable for the retention period; copy them to Blob with `azcopy copy` using the same relative path layout (confirm the layout Airflow expects before bulk copying); or archive them without making them visible in the UI.

Operations. Alert on upload errors in task logs and on Blob 403 and 5xx metrics; review storage cost monthly; review the identity role assignments quarterly.

Rollback. Set remote logging to false and restart the affected components. Local logs were kept, so recent runs stay readable; logs already in Blob are retained but not shown.

14.4 Phase 2 acceptance criteria

Item	Success criteria	Soak
NFS 4.1 DAG share	Parse time and scheduler loop time meet the target set from the v3 SMB baseline; no stale-file or permission errors; both shares in sync; deployer pipeline automated	7 days with no storage-related incident
Blob remote logging	Logs readable in the UI for every task in the soak; no unexplained upload errors; access by identity only; lifecycle policy confirmed	14 to 30 days

14.5 After Phase 2: retire the SMB shares

- [ ] Stop deploying releases to the SMB DAG share; keep it read-only for 30 days, then delete it.
- [ ] Keep the log share read-only until the retention period for old logs has passed, then delete it.
- [ ] Remove the SMB PVs, PVCs and chart values; update runbooks, diagrams and on-call notes.
- [ ] Update cost tracking: premium NFS capacity and Blob capacity replace the two SMB shares.

14.6 Later options

- A custom DAG bundle that fetches a pinned DAG package version from Nexus gives Airflow-native bundle versioning without Git, and fits your release rules. Treat it as a design task, not a configuration change.

15. Risk register

The highest risks are a slow or failed schema migration, DAG behavior changes that tests did not catch, and duplicate or missed runs around the cutover and any rollback.

#	Risk	Likelihood	Impact	Mitigation	Owner
1	Migration takes longer than the window	Medium	High	Trim history on the clone; rehearse on prod-sized clone; window = 1.5x measured; abort path B	DBA
2	Migration fails or leaves a half-applied schema	Low	High	Run from inside the cluster; clone-only; drop and redo; never repair in place	DBA
3	Direct 2.10.5 to 3.3.2 path unsupported or flawed	Medium	Medium	Prove the direct path in dev; hop through 2.11.x on the clone only if direct fails validation	Platform
4	DAGs break on removed or changed features	High	High	Ruff rules; manual review; fixes shipped to v2 first; tier-0 regression in UAT	DAG owners
5	Direct DB access in DAG code fails under the new execution model	Medium	High	Search for session use and raw ORM queries; refactor to supported interfaces	DAG owners
6	Changed schedule defaults cause missed or extra runs	Medium	High	Set `catchup`, `start_date` and timetable explicitly; controlled unpause; owner confirms first runs	DAG owners
7	Auth or role model differs and blocks or over-permits users	Medium	Medium	FAB provider decision early; role mapping tests; security sign-off	Security
8	External API callers break (new API version and token flow)	High	Medium	Inventory callers; migrate in UAT; hand-over checklist at cutover	Platform

#	Risk	Likelihood	Impact	Mitigation	Owner
9	Fernet key mismatch makes connections undecryptable	Low	High	Reuse the v2 key; decrypt tests before go-live	Platform
10	Duplicate side effects after a rollback	Medium	High	Rollback restarts v2 with all DAGs paused; owner-approved reconciliation	DAG owners
11	DAG parsing is slow on SMB share	Medium	Medium	Measure in UAT; NFS in Phase 2; avoid scale-out of DAG count during window	Cloud
12	v3 DAG share and v2 share drift apart	Medium	High	Single pipeline deploys to both; checksum job; freeze	Release
13	v2 and v3 schedulers run together	Low	Very high	Runbook gate; v3 uses separate DB and Redis; checklist signed at T0	Change owner
14	Nodepool quota or capacity is insufficient for two stacks	Medium	Medium	Quota request in week 1; confirm at T-1d	Cloud
15	PostgreSQL server IOPS or storage exhausted during clone and migrate	Low	High	2.5x storage rule; option B for prod if needed	DBA
16	Custom plugin or UI extension does not work	Medium	Medium	Plugin inventory; early test; plan for rewrite or retirement	Platform
17	Provider version incompatibility	Medium	Medium	Use the 3.3.2 constraints file; test each connection class	Platform
18	Deferred or long-running tasks span the cutover	Medium	Medium	Drain policy; tier-0 owner decision; avoid cutover during long jobs	Change owner
19	Observability gaps: dashboards and alerts keyed to old components	High	Medium	Update and test alerts for api-server and dag-processor in UAT	Operations
20	Team fatigue or unclear authority in a long window	Medium	Medium	Named roles; shift handover; fixed decision points; reserve for rollback	Change owner
21	Python 3.11 to 3.13 move breaks a DAG dependency or custom library	Medium	High	CI matrix on 3.11 and 3.13; resolve against the 3.13 constraints file; check wheels for native packages; regression in UAT (section 5.5)	DAG owners, Platform

#	Risk	Likelihood	Impact	Mitigation	Owner
22	PgBouncer pool saturates under v3 load (the api-server handles every task state change)	Medium	Medium	Separate pool for v3; load test with wait metrics; raise pool size to 80 to 100 if clients wait and the SKU has headroom	DBA

16. Appendices

A. Isolation checklist (sign before every cutover and rehearsal)

Resource	v2	v3	Verified by
Namespace	existing	`airflow-v3`	Platform
Nodepool	existing	new, tainted	Cloud
Metadata database	`airflow`	`airflow_v3` (clone)	DBA
DB roles and passwords	existing	new	DBA
Celery result backend	v2 DB	v3 DB	Platform
Redis instance and credentials	existing	new	Cloud
DAG share and PVC	existing	new	Cloud
Logs path	existing	new	Cloud
Fernet key	key K	same key K	Platform
JWT secret, API secret key	n/a	new	Platform
Hostname and ingress	production	test host, then production at cutover	Cloud
SSO redirect URIs	existing	added	Security
Image	v2 digest	v3 digest	Platform
Schedulers running at the same time	must be zero	must be zero	Change owner

B. Command reference

Purpose	Command (placeholders in angle brackets)
Lint configuration	`airflow config lint`
Update configuration	`airflow config update`
Ruff scan of DAGs	`ruff check --preview --select AIR30 dags/`
Trim history	`airflow db clean --clean-before-timestamp "<ts>" --skip-archive --yes`
Migrate	`airflow db migrate`
Check migrations	`airflow db check-migrations --migration-wait-timeout 60`
Check DB connection	`airflow db check`
Scheduler health	`airflow jobs check --job-type SchedulerJob`
dag-processor health	`airflow jobs check --job-type DagProcessorJob`
Triggerer health	`airflow jobs check --job-type TriggererJob`
List DAGs	`airflow dags list`
Pause or unpause	`airflow dags pause <dag_id>` / `airflow dags unpause <dag_id>`
Import errors	`airflow dags list-import-errors`
Test a connection	`airflow connections test <conn_id>`
Scale a workload	`kubectl scale deploy/<name> -n <ns> --replicas=<n>`
Helm release state	`helm list -n <ns>`; `helm get values <release> -n <ns> -o yaml`
On-demand DB backup	`az postgres flexible-server backup create ...`
Active DB sessions	`SELECT * FROM pg_stat_activity WHERE datname = '<db>';`

C. Configuration keys to set or review in v3

The names below are the usual Airflow environment variable forms. Confirm each against the 3.3.2 configuration reference and the Helm chart's `values.yaml` before use.

Setting	Environment variable	Note
Fernet key	`AIRFLOW__CORE__FERNET_KEY`	Same as v2
Metadata DB	`AIRFLOW__DATABASE__SQL_ALCHEMY_CONN`	v3 database only; pooled port (6432) at runtime, direct port (5432) for migration and admin jobs
Broker	`AIRFLOW__CELERY__BROKER_URL`	New Redis
Celery result backend	`AIRFLOW__CELERY__RESULT_BACKEND`	v3 database
JWT secret	`AIRFLOW__API_AUTH__JWT_SECRET`	New; shared across components
API secret key	`AIRFLOW__API__SECRET_KEY`	New; shared across api-server replicas
Execution API URL	`AIRFLOW__CORE__EXECUTION_API_SERVER_URL`	api-server Service URL ending in `/execution/`
Auth manager	`AIRFLOW__CORE__AUTH_MANAGER`	FAB auth manager class if using the FAB provider
Catchup default	`AIRFLOW__SCHEDULER__CATCHUP_BY_DEFAULT`	Default is now false; set DAGs explicitly anyway
Public base URL	`AIRFLOW__API__BASE_URL`	Production hostname after cutover

D. Open items to verify before the dev build

#	Item	Why it matters
1	Python 3.13 compatibility of every DAG dependency, custom library and provider extra (Airflow 3.3.2 itself supports Python 3.10 to 3.14)	Image build; provider compatibility

#	Item	Why it matters
2	Helm chart version that supports Airflow 3.3.2, and its value names	Release definition
3	Confirm in the 3.3.2 documentation that 2.10.5 is a supported starting version for a direct migration; hop through 2.11.x only if it is not	Section 7.6 decision
4	Significant changes in each release from 3.0 through 3.3.2, which all apply to a 2.x upgrader	Behavior changes not covered here
5	FAB provider version and options for RBAC, LDAP or OAuth; how API clients authenticate	Auth and external callers
6	Behavior of `airflow db clean` regarding the latest run per DAG	Scheduling continuity after trim
7	Permissions for `CREATE DATABASE ... TEMPLATE` on Azure Database for PostgreSQL Flexible Server	Clone option A
8	Technique for marking missed intervals complete in v2 during reconciliation	Rollback reconciliation drill
9	Celery queue names and Redis tier behavior	Drain checks
10	Deferred task behavior across the migration	Cutover drain policy
11	Plugin and UI extension compatibility	Section 8, test 16
12	Behavior of manual runs on paused DAGs	Smoke DAG procedure
13	PgBouncer pool mode, max_client_conn, idle timeout and minimum pool size; server SKU; v2 peak connections	Pool sizing and the clone step

E. Assumptions

- Executor is CeleryExecutor with Redis; the official Apache Airflow Helm chart is used.
- PostgreSQL 17 runs on Azure Database for PostgreSQL Flexible Server; point-in-time restore is enabled.
- The Nexus release pipeline can be extended to deploy the same DAG version to a second share.

- Dev, UAT and production share the same architecture; sizes differ.
- Timelines are indicative and will be replaced with measured values from the rehearsals.
- DAG count, DB size, provider list and plugin inventory are not yet known and drive the real window.

F. Sources

Consulted through search results; open the pages for the full text before relying on specifics.

- Upgrading to Airflow 3 (official guide)
- Airflow 3.3.2 release
- Airflow release notes